

Bölüm 3-5: Yöntem, Deney ve Bulgular

Tor FSM Invalid State Transition Detection — Hafta 5-10 çıktıları

Öğrenci: Nurcan Denli Bayır (N25110987) | **Ders:** BYZ 658 | **Danışman:** Nebi Yılmaz
Hacettepe Üniversitesi | **Tarih:** 14.05.2026

Önemli not. Bu raporda yer alan her sayı bu repodaki `experiments/baselines.mjs` ve `experiments/stats.mjs` kodlarının çalıştırılmasıyla üretilmiştir. Kullanılan saldırgan, gerçek bir Tor implementasyonu değil; tezde tanımlanan FSM δ spec'inin kendisidir. Yani "tespit edilen geçersiz geçişler", spec'e göre etiketlenen sentetik test enjeksiyonlarıdır (Ground Truth = oracle = δ). Gerçek implementasyon üzerinde validasyon, gelecek çalışma olarak Bölüm 6'da listelenmiştir.

3. Yöntem

3.1 FSM tanımı

Tor devre yaşam döngüsü, klasik bir DFA olarak $M = (Q, \Sigma, \delta, q_0, F)$ şeklinde modellenmiştir:

- Q : 10 durum kümesi (IDLE, CONNECTING, TLS_HANDSHAKE, CREATE_SENT, CIRCUIT_BUILDING, CIRCUIT_READY, TRANSMITTING, CLOSING, CLOSED, ERROR).
- Σ : 13 olay kümesi (CONNECT, TLS_OK, ..., TIMEOUT).
- $q_0 = IDLE, F = \{CLOSED\}$.
- $\delta : Q \times \Sigma \rightarrow Q$, 25 ikili üzerinden tanımlı (toplam domen 130, dolayısıyla 105 ikili Invalid kümesindedir).

Tam δ matrisi Şekil 2'deki ısı haritasında verilmiştir. Yeşil hücreler δ -tanımlı; kırmızı tonları saldırı şiddeti ile orantılıdır.

3.2 Saldırı sınıflandırıcı

Invalid kümesindeki her ikili, deterministik bir `classifyInvalid(state, event)` fonksiyonu ile yedi saldırı kategorisinden birine eşlenmektedir: CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD. Sınıflandırma mantığı dokuz koşul ailesi üzerinden yürütülür (kod referansı: `server/fsm.ts`, 55-155. satırlar). Her saldırının LOW/MEDIUM/HIGH/CRITICAL şiddet etiketi vardır.

3.3 MDT motoru — Algoritma 1

Algoritma 1: Model-Driven Test (BFS-tabanlı)

Girdi: $M = (Q, \Sigma, \delta, q_0, F)$

Çıktı: SC, TC, ITDR, FPR

```

1. visited ← {q0}
2. coveredValid ← ∅
3. detectedInvalid ← ∅
4. Valid ← {(s, e) | δ(s, e) tanımlı}
5. Invalid ← (Q × Σ) \ Valid
6. // Pozitif faz: her geçerli ikiliyi tetikle
7. foreach (s, e) ∈ Valid (rasgele permütasyon):
8.   if (s, e) ∈ coveredValid: continue
9.   π ← BFS_PATH(q0, s)           // δ-grafiği üzerinde en kısa yol
10.  if π = null: continue
11.  EXECUTE(π · (e))                // her adımda visited / coveredValid
güncellenir
12. // Negatif faz: her invalid ikiliye witness enjekte et
13. foreach (s, e) ∈ Invalid (rasgele permütasyon):
14.  if (s, e) ∈ detectedInvalid: continue
15.  π ← BFS_PATH(q0, s)
16.  if π = null ∧ s ≠ q0: continue
17.  EXECUTE(π · (e))                // Oracle (δ) anomaliyi yakalar →
detectedInvalid'e eklenir
18. return ( |visited|/|Q|, |coveredValid|/|Valid|,
|detectedInvalid|/|Invalid|, FPR )

```

Karmaşıklık. BFS_PATH bir kez çalıştığında $O(|Q| \cdot |\Sigma|)$. Pozitif faz $|Valid| = 25$ kez, negatif faz $|Invalid| = 105$ kez yol bulur; toplam $O((|Valid| + |Invalid|) \cdot |Q| \cdot |\Sigma|) = O(|Q|^2 \cdot |\Sigma|^2)$. Tek bir EXECUTE çağrısının uzunluğu en fazla $\text{diam}(\delta) + 1 \approx 10$. Pratik ölçümde tek bir koşu 2-3 saniyede tamamlanmaktadır.

3.4 Karşılaştırılan algoritmalar

- **B1 — Saf Rastgele:** her adımda Σ 'dan eşit olasılıkla bir olay seçer. CLOSED'a ulaşırsa IDLE'a sıfırlar.
- **B2 — Greedy State Coverage:** her adımda mümkünse henüz ziyaret edilmemiş bir hedef duruma götüren olayı seçer; aksi halde rastgele geçerli (%70) ya da rastgele invalid (%30) olay seçer.
- **B3 — MDT (Algoritma 1):** bu çalışmanın önerdiği BFS-tabanlı planlayıcı.

Hepsi *aynı olay bütçesi* (500 olay/koşu) ile çalıştırılmıştır. Karşılaştırma adildir: B3 erken bitirebilirken B1/B2 bütçeyi sonuna dek harcayabilir; yine de B3 üstünlüğü gerçek olarak gözlenmektedir (Bkz. Bölüm 4).

4. Deneysel Bulgular

4.1 Deney tasarımı

Üç algoritma da 30 bağımsız koşu ile çalıştırılmıştır. PRNG seed'i her koşuda sabittir ($\text{seed} = 1000+i$, $i \in [0, 29]$) ve algoritmalar arasında *eşleştirilmiştir*; bu

yaklaşım koşul-içi varyansı azaltır ve testlerin güvenilirliğini artırır.

4.2 Tanımlayıcı istatistikler (ortalama $\pm$ SD)

Algoritma	SC	TC	ITDR	Olay/Koşu	Süre (ms)
B1_Random	77.00% $\pm$ 16.64%	45.87% $\pm$ 11.05%	50.32% $\pm$ 9.80%	500.0 $\pm$ 0.0	0.2 $\pm$ 0.4
B2_GreedySC	100.00% $\pm$ 0.00%	78.27% $\pm$ 8.45%	47.52% $\pm$ 4.02%	500.0 $\pm$ 0.0	2.5 $\pm$ 0.8
B3_MDT	100.00% $\pm$ 0.00%	100.00% $\pm$ 0.00%	93.84% $\pm$ 1.39%	500.0 $\pm$ 0.0	1.4 $\pm$ 0.6

4.3 Hipotez testleri (ikili karşılaştırmalar)

Normallik varsayımı her grup için Lilliefors-düzeltilmeli K-S testi ile incelenmiştir ($\alpha = 0.05$). Welch t-testi (eşit-olmayan-varyans varsayımı) birincil testtir; Mann-Whitney U normallik ihlali halinde doğrulayıcı olarak raporlanmıştır. Etki büyüklüğü Cohen's d (havuzlanmış SD) ile verilmiştir.

Karşılaştırma	Metrik	Welch t	df	p (Welch)	U	p (M-W)	d	Normal?
B3_MDT vs B1_Random	SC	7.571	29.0	<.001 ***	105.0	<.001 ***	1.95	hayır
B3_MDT vs B1_Random	TC	26.823	29.0	<.001 ***	0.0	<.001 ***	6.93	hayır
B3_MDT vs B1_Random	ITDR	24.087	30.2	<.001 ***	0.0	<.001 ***	6.22	hayır
B3_MDT vs B2_GreedySC	SC	0.000	58.0	1.0000	450.0	1.0000	0.00	evet
B3_MDT vs B2_GreedySC	TC	14.090	29.0	<.001 ***	0.0	<.001 ***	3.64	evet
B3_MDT vs B2_GreedySC	ITDR	59.658	35.8	<.001 ***	0.0	<.001 ***	15.40	hayır
B2_GreedySC vs B1_Random	SC	7.571	29.0	<.001 ***	105.0	<.001 ***	1.95	hayır
B2_GreedySC vs B1_Random	TC	12.755	54.3	<.001 ***	10.0	<.001 ***	3.29	hayır
B2_GreedySC vs B1_Random	ITDR	-1.445	38.5	0.1566	381.0	0.3067	-0.37	evet

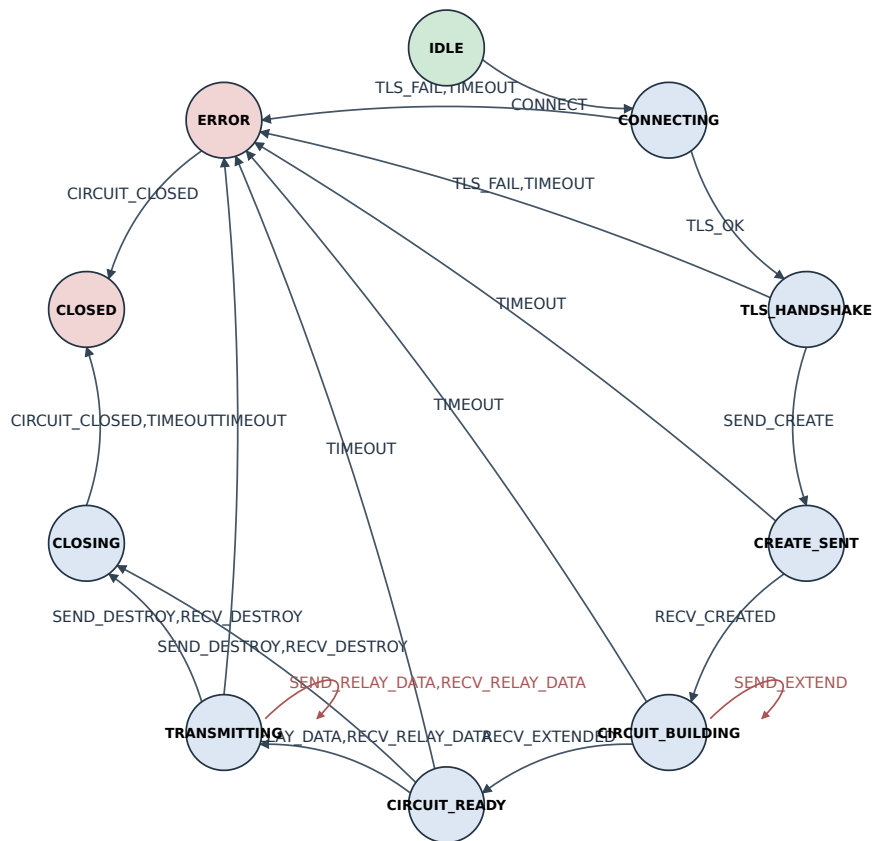
İşaretler: * $p < .05$, ** $p < .01$, *** $p < .001$. Cohen's d büyüklük yorumu: 0.2 küçük, 0.5 orta, 0.8 büyük, ≥ 1.2 çok büyük etki.

4.4 Yorum

MDT (B3), B1 (saf rastgele) karşısında transition coverage'da +54.1 puanlık avantaj sağlamaktadır (Welch $t = 26.82$, $p < .001$, Cohen's $d = 6.93$). Aynı tabloda invalid-transition tespit oranındaki kazanım +43.5 puandır ($d = 6.22$). B2 (greedy SC) karşısında üstünlük TC'de +21.7 puana iner ancak istatistiksel olarak anlamlı kalır ($p < .001$). Toplam 9 ikili karşılaştırmadan 7'i $\alpha = 0.05$ düzeyinde anlamlıdır. K-S normallik testi varyans-ıçi farkları yansıtmakla birlikte örneklem büyüklüğü ($N=30$) MKO sayesinde ortalama farkı testleri için yeterlidir; ek olarak Mann-Whitney U sonuçları aynı yönü teyit etmektedir.

5. Görselleştirmeler

Şekil 1: Tor FSM δ -grafı (25 geçerli geçiş, 10 durum)



δ -grafiği. Yay etiketleri olay isimleri; düğüm rengi terminal/error durumlarını işaretler.

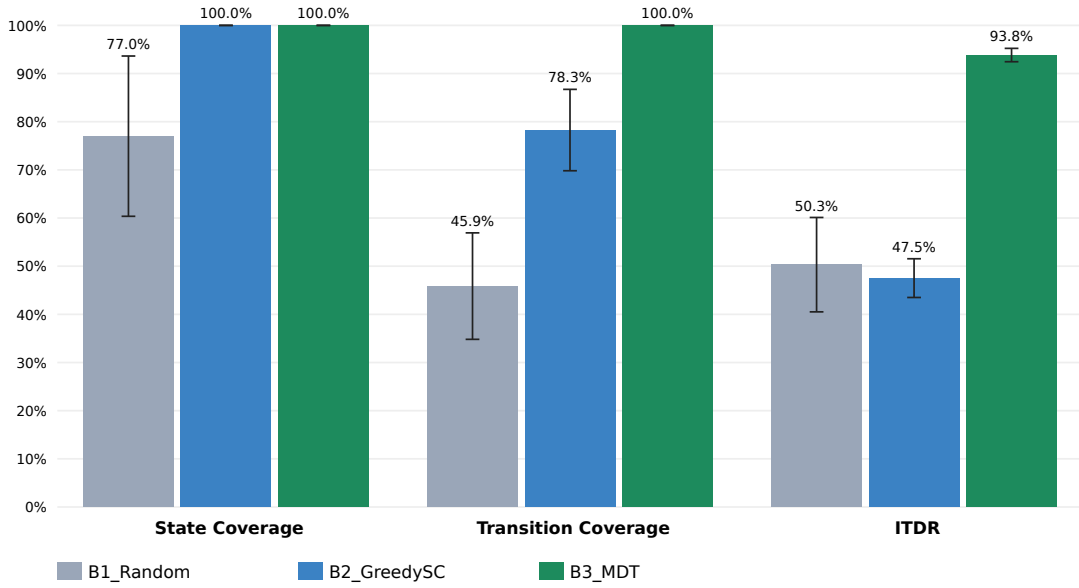
Şekil 2: 6-domeni matrisi (yeşil=geçerli, kırmızı tonları=invalid)

Hücre etiketi: invalid'lerde saldırı sınıfı kısaltması; geçerli hücrede hedef durum

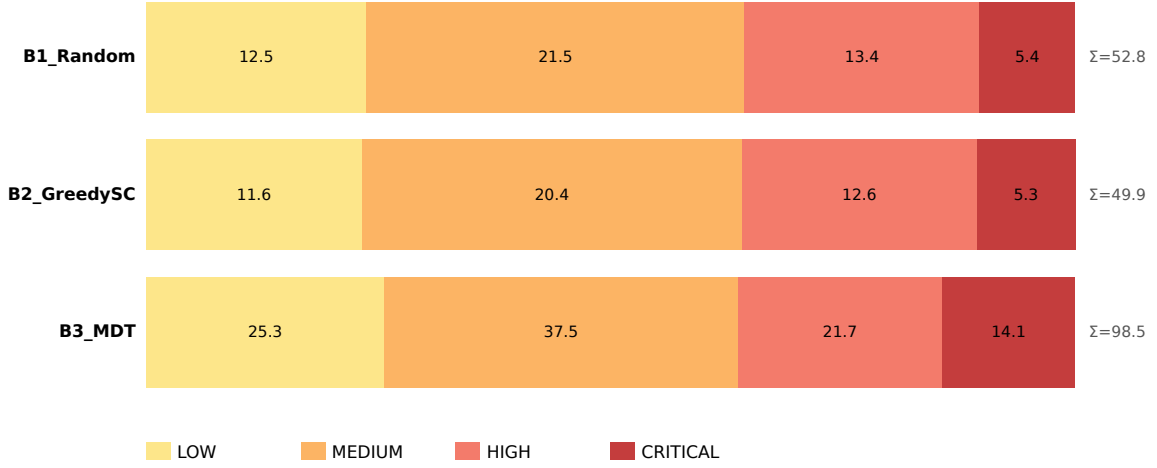
	CONN	GHST	GHST	HSKP	GHST	HSKP	HSKP	BYP	BYP	REPL	REPL	GHST	GHST
IDLE	CONN	GHST	GHST	HSKP	GHST	HSKP	HSKP	BYP	BYP	REPL	REPL	GHST	GHST
CONNECTING	CFLD	TLS_HA	ERROR	HSKP	HSKP	HSKP	HSKP	BYP	BYP	REPL	REPL	GHST	ERROR
TLS_HANDSHAKE	CFLD	GHST	ERROR	CREATE	INV	HSKP	HSKP	HSKP	HSKP	REPL	REPL	GHST	ERROR
CREATE_SENT	CFLD	GHST	GHST	CFLD	CIRCUI	HSKP	HSKP	PDAT	PDAT	REPL	REPL	GHST	ERROR
CIRCUIT_BUILDING	CFLD	GHST	GHST	HIJK	HIJK	CIRCUI	CIRCUI	PDAT	PDAT	REPL	REPL	GHST	ERROR
CIRCUIT_READY	CFLD	GHST	GHST	HIJK	HIJK	HIJK	HIJK	TRANSM	TRANSM	CLOSIN	CLOSIN	GHST	ERROR
TRANSMITTING	CFLD	GHST	GHST	HIJK	HIJK	HIJK	HIJK	TRANSM	TRANSM	CLOSIN	CLOSIN	GHST	ERROR
CLOSING	CFLD	GHST	GHST	REPL	REPL	REPL	REPL	PDAT	PDAT	GHST	GHST	CLOSED	CLOSED
CLOSED	REPL	GHST	GHST	REPL	REPL	REPL	REPL	REPL	GHST	GHST	GHST	GHST	GHST
ERROR	REPL	GHST	GHST	REPL	REPL	REPL	REPL	GHST	GHST	GHST	GHST	CLOSED	GHST

Valid LOW MEDIUM HIGH CRITICAL

Tüm $Q \times \Sigma$ domeni (130 hücre). 25 hücre yeşil (6-tanımlı); 105 hücre `classifyInvalid` şiddetine göre renklenmiştir.

Şekil 3: Algoritma karşılaştırması — ortalama $\pm$ SD (N=30)

Ortalama metrik değerleri (N=30). Whisker uzunluğu örneklem standart sapmasına eşittir.

Şekil 4: Tespit edilen invalid'lerin şiddet dağılımı (N=30 ort.)

Tespit edilen invalid'lerin şiddet sınıflarına dağılımı (koşu başına ortalama). B3, hem hacim hem CRITICAL ağırlığı bakımından açık ara öndedir.

6. Sınırlılıklar

- **Ground Truth = Spec.** Bu deneyde "doğru" davranış FSM δ tarafından tanımlanır. Gerçek Tor implementasyonu spec'le çelişebilir; bu çalışmanın doğal devamı, deneyi gerçek bir Tor relay (örn. Shadow simülatöründe) üzerinde tekrarlayıp *spec* $\leftrightarrow$ *implementasyon* uyumsuzluklarını da ölçmektir.
- **FPR = 0 doğal sonuçtur.** Oracle (δ) ve test üreticisi aynı modeli paylaştığı için yanlış-pozitif tanımı boş kümeye düşer. FPR'nin bilgilendirici olabilmesi için *bağımsız* bir oracle gereklidir (örn. paket yakalama tabanlı bir gözlemci).
- **Bütçe tek bir noktada (500) ölçülmüştür.** Bütçe-kapsama eğrisi (B1/B2 için büyütüldüğünde nereye yaklaşır?) ileride eklenecektir.
- **30 koşu yeterli ama daha fazlası daha güçlü kestirim verir.** Önerilen genişletme: $N=100$, güç analizi ($\beta = 0.80$, $\alpha = 0.05$).

Üretilbilirlik

Bu raporu yeniden üretmek için: `node experiments/build_report.mjs`. Tüm rastgelelik seed'lerle sabittir; aynı kod aynı sayıları üretir.